

FEMU Experiment

WAF of EXT4 and F2FS

2026. 08. 19

Presentation by 김정민, 정다원

kyrios3929@dankook.ac.kr

defimedjustice1220@gmail.com

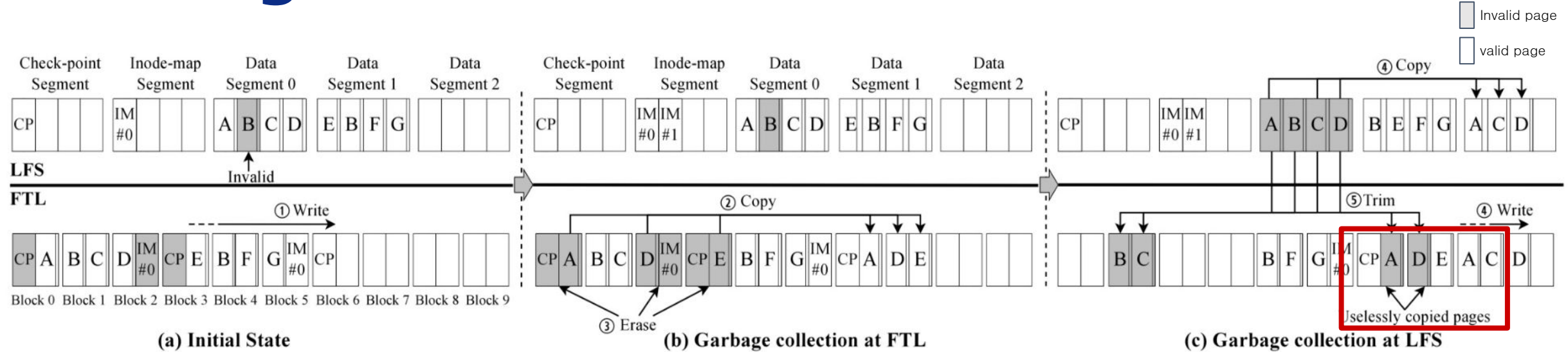
Contents

1. Motivation
2. Background
3. Definition of WAF
4. Experiment Method
5. Workload
6. Expected Result
7. Experimental result
8. Analysis
9. Conclusion & Future Work

1. Motivation

- Flash에서는 **overwrite**가 어렵기 때문에 데이터 갱신, GC 과정에서 추가적인 **Write**로 인한 **Write Amplification**이 발생한다
- F2FS는 **segment cleaning**을 통해 Flash에 효율적으로 데이터를 기록하고, 불필요한 **Write**를 줄임
- 하지만 AMF 논문에서는 F2FS의 **FS-level Cleaning** 이후에도, **FTL**이 다시 **GC**를 수행될 수 있어 Flash 관리가 중복될 수 있음을 지적
- 따라서 F2FS에서 실제로 **Workload**에 따라 **Write Amplification**이 어떻게 발생하는지 실험을 통해 확인하고자 한다

2. Background - F2FS



LFS와 FTL의 이중 GC

- LFS는 Segment 단위로, FTL은 Flash Block 단위로 각각 GC 수행.
- 두 계층의 data copy가 각각 따로 진행된다.
- 추가 Write로 인한 WAF가 증가한다.

2. Background - FEMU

QEMU: 가상 하드웨어 환경을 제공

FEMU: 가상 하드웨어 환경 위에서 BlackBox등의 가상 SSD의 동작을 구현

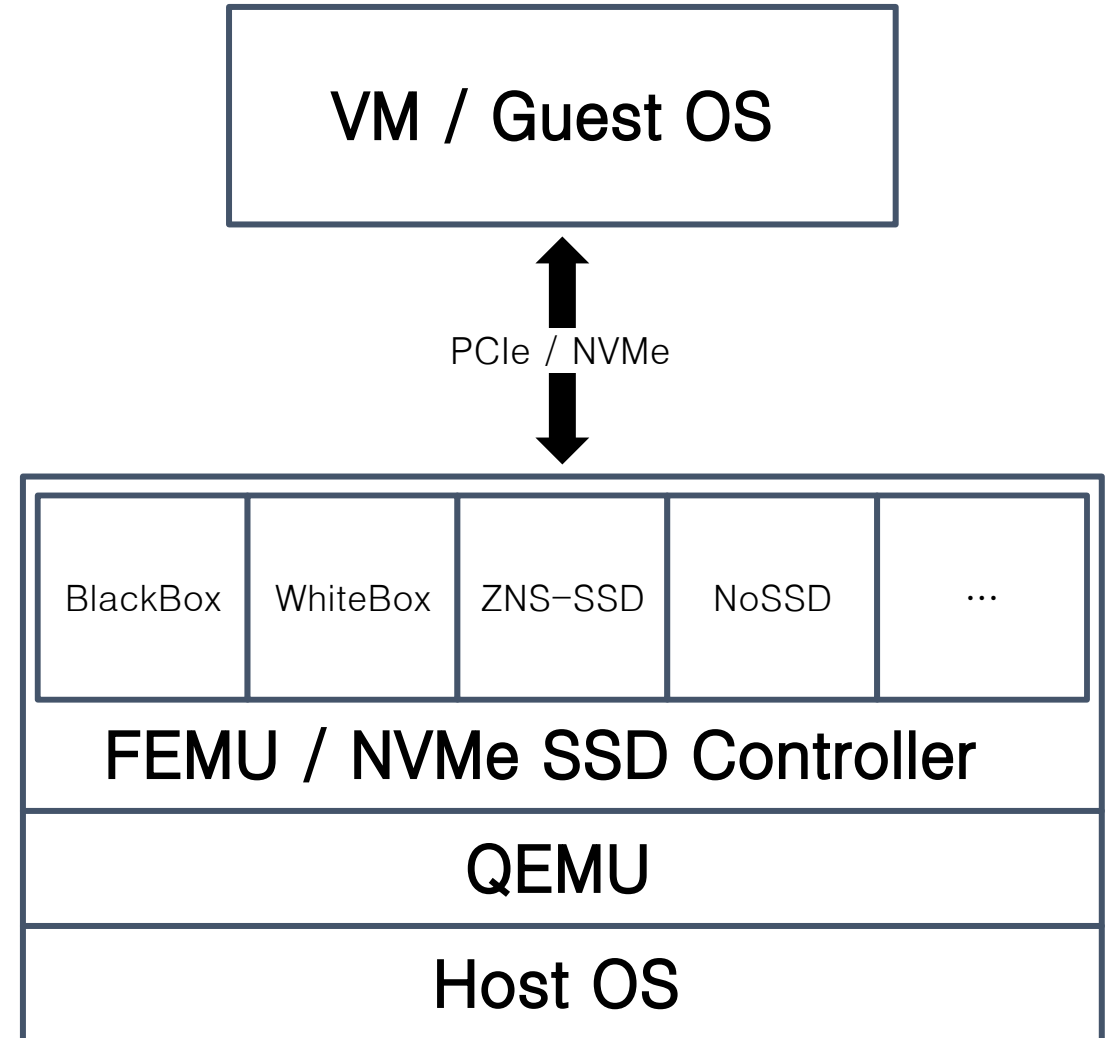
VM / Guest OS:

구현된 가상 하드웨어를

PCIe (인터페이스)와 NVMe (프로토콜)로 인식

NVMe SSD Controller :

Guest OS로 부터 온 명령을 받아 FEMU의 SSD 동작으로 변환



3. Definition of WAF

- $FS\ WAF = \frac{FS\text{가 실제로 } FTL\text{에 요청한 용량}}{Host\text{가 요청한 쓰기량}}$: Host의 쓰기 요청에 대해 FS에서 발생하는 추가 Write
- $FTL\ WAF = \frac{FTL\text{의 GC 포함 실제 } Flash\text{ 쓰기량}}{FS\text{이 } FTL\text{에 전달한 쓰기량}}$: FS의 쓰기 요청에 대해 FTL GC로 발생하는 추가 Flash Write
- $TOTAL\ WAF = \frac{\text{최종 } Flash\text{에 기록된 전체 용량}}{Host\text{ 쓰기량}}$: Host의 쓰기 요청에 대해 최종 Flash Write까지 발생한 전체 Write Amplification

→ 논문에서 사용하는 정의는 Total WAF

3. Definition of WAF

- $$\frac{FS \text{ 쓰기량}}{\text{사용자 요청량}} = \frac{User \text{ Page Write} \times Page \text{ Size}}{\text{사용자 요청량}}$$
- $$\frac{Flash \text{ Write Size}}{FS \text{ Write Size}} = \frac{(GC \text{ Page Write} + User \text{ Page Write}) \times Page \text{ Size}}{User \text{ Page Write} \times Page \text{ Size}}$$
- $$\frac{(GC \text{ Page Write} + User \text{ Page Write}) \times Page \text{ Size}}{\text{사용자 요청량}} = FTL \text{ WAF} \times FS \text{ WAF}$$
- User Page Write : Host의 Write 요청을 처리하기 위해 발생한 Page Write
- GC Page Write : GC 과정에서 Vaild Page를 이동하며 발생한 추가 Page Write
- WAF = 1 : Host가 요청한 만큼만 Flash에 Write
- WAF가 클수록 Flash Write가 증가하여 성능 및 Flash 수명이 감소

4. Experiment Method

FemuCtrl 구조체 : **NVMe** 컨트롤러가 보는 FEMU SSD의 상태

ssd 구조체 : **FTL** 동작에서 SSD를 대표하는 구조체

두 구조체는 상호 참조하는 관계

4. Experiment Method

구상한 Page 측정 방법 :

guest OS가 NVMe를 통해서 요청을 보내면, **NVMe 컨트롤러**가 받아서 **Page** 상태를 출력하는 방식.

따라서 FemuCtrl 구조체에 **Page** 정보를 추가하였다.

user_write_pages : guest OS의 요청으로 인해 쓴 Page의 수

gc_write_pages : Garbage collector로 인해 쓴 Page의 수

```
typedef struct FemuCtrl {  
    PCIDevice      parent_obj;  
    MemoryRegion   iomem;  
    MemoryRegion   ctrl_mem;  
    NvmeBar        bar;  
}
```

(생략)

```
uint64_t user_write_pages;  
uint64_t gc_write_pages;  
} FemuCtrl;
```

4. Experiment Method

FTL 동작 구현인 FTL.c

ssd_write() : quest OS의 request를 받아
해당하는 각 page에 쓰는 함수

user_write_pages 증가 구문 추가

```
static uint64_t ssd_write(struct ssd *ssd, NvmeRequest *req)  
{  
    uint64_t lbn = req->lbn;
```

입력은 Request 단위로 들어옴

```
    for (lbn = start_lbn; lbn <= end_lbn; lbn++) {
```

```
        /* new write */  
        ppa = get_new_page(ssd);  
        /* update maptbl */  
        set_maptbl_ent(ssd, lbn, &ppa);  
        /* update rmap */  
        set_rmap_ent(ssd, lbn, &ppa);
```

```
        mark_page_valid(ssd, &ppa);  
        ssd->n->user_write_pages++; for문 안, request에 해당하는 모든 page에 대해 카운트
```

4. Experiment Method

FTL 동작 구현인 FTL.c

gc_write_page() : 다른 위치로 복사할
PPA를 받아 기존 page를 비활성화하고,
새 page에 copy하는 부분

gc_write_pages 증가 구문 추가

```
/* move valid page data (already in DRAM) from victim line to a new page */
static uint64_t gc_write_page(struct ssd *ssd, struct ppa *old_ppa)
{
    struct ppa new_ppa;
    struct nand_lun *new_lun;
    uint64_t lpn = get_rmap_ent(ssd, old_ppa);

    ftl_assert(valid_lpn(ssd, lpn));
    new_ppa = get_new_page(ssd);
    /* update maptbl */
    set_maptbl_ent(ssd, lpn, &new_ppa);
    /* update rmap */
    set_rmap_ent(ssd, lpn, &new_ppa);
    if (exp_lpn_watched(lpn)) {
        exp_watch_blk[new_ppa.g.blk] = 1; /* track the new block too */
        EXP_LOG("[GC_MOVE] lpn=%lu " PPA_FMT " -> " PPA_FMT "\n",
                lpn, PPA_ARG(old_ppa), PPA_ARG(&new_ppa));
    }

    mark_page_valid(ssd, &new_ppa);
    ssd->n->gc_write_pages++;
}
```

4. Experiment Method

nvme_get_log : Guest OS의 nvme get-log 로 호출되는 함수

이때 Log ID 인자에서 새로 정의한 0xCA를 받게 되면, Page 상태를 **Host OS**에 출력

```
static uint16_t nvme_get_log(FemuCtrl *n, NvmeCmd *cmd)
{
```

```
    switch (lid) {
    case NVME_LOG_ERROR_INFO:
        return nvme_error_log_info(n, cmd, len);
    case NVME_LOG_SMART_INFO:
        return nvme_smart_info(n, cmd, len);
    case NVME_LOG_FW_SLOT_INFO:
        return nvme_fw_log_info(n, cmd, len);
    case NVME_LOG_CMD_EFFECTS:
        return nvme_cmd_effects(n, cmd, csi, len, off);
    case NVME_LOG_ENDGRP:
        return nvme_endgrp_info(n, len, off, cmd);
    case NVME_LOG_FDP_CONFS:
        return nvme_fdp_confs(n, lspci, len, off, cmd);
    case NVME_LOG_FDP_RUH_USAGE:
        return nvme_fdp_ruh_usage(n, lspci, len, off, cmd);
    case NVME_LOG_FDP_STATS:
        return nvme_fdp_stats(n, lspci, len, off, cmd);
    case NVME_LOG_FDP_EVENTS:
        return nvme_fdp_events(n, lspci, len, off, cmd);
    case NVME_LOG_FEMU_WRITE:
        printf("user_page_write: %lu\n", n->user_write_pages);
        printf("gc_page_write: %lu\n", n->gc_write_pages);
        return NVME_SUCCESS;
    default:
        if (n->ext_ops.get_log) {
            return n->ext_ops.get_log(n, cmd);
        }
        return NVME_INVALID_LOG_ID | NVME_DNR;
    }
}
```

```
root1@root1-Z390-GAMING-X:~/FEMU/hw$ grep "NVME_LOG_FEMU_WRITE" -n ./femu/nvme.h
997:     NVME_LOG_FEMU_WRITE = 0xCA,
root1@root1-Z390-GAMING-X:~/FEMU/hw$
```

4. Experiment Method : 실제 TEST

```
root1@root1-Z390-GAMING-X:~/FEMU/build-femu$ ./run-blackbox-ext4.sh
-device femu,devsz_mb=12288,namespaces=1,femu_mode=1,secsz=512,secs_per_pg=8,pgs_per_blk=256,blks_per_pl=256,pls_per_lun=1,luns_per_ch=8,nchs=8,pg_rd_lat=40000,pg_w
r_lat=200000,blk_er_lat=2000000,ch_xfer_lat=0,gc_thres_pcent=75,gc_thres_pcent_high=95
[sudo] password for root1:
VNC server running on 127.0.0.1:5900
user_page_write: 1
gc_page_write: 0
```

HostOS

```
femu@fvm:~$ dd if=/dev/zero of=/tmp/4k.bin bs=4096 count=1
1+0 records in
1+0 records out
4096 bytes (4.1 kB, 4.0 KiB) copied, 9.2892e-05 s, 44.1 MB/s
femu@fvm:~$ sudo nvme write /dev/nvme0n1 \
> --start-block=0 \
> --block-count=0 \
> --data-size=4096 \
> --data=/tmp/4k.bin
[sudo] password for femu:
write: Success
femu@fvm:~$ sudo nvme get-log "/dev/nvme0n1" --log-id=0xCA --log-len=4
Device:nvme0n1 log-id:202 namespace-id:0xffffffff
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
0000: 00 00 00 00                                     "...."
femu@fvm:~$
```

GuestOS

5. Workload

Host CPU	Intel Core i7-8700 @ 3.20 GHz	Sector Size	512 B
Host CPU cores	(6 cores / 12 threads)	Page Size	4KiB (Sector 8개)
Guest vCPU	4 vCPUs	Block Size	1MiB (Page 256개)
Guest Memory	4 GiB	Blocks per Plane	256
Guest OS	Ubuntu 20.04.1 LTS (Focal Fossa)	Planes per LUN	1
Guest Kernel	Linux 5.4.0-216-generic	LUNs per Channel	8
		Channels	8
		Physical NAND capacity	16 GiB
		User-visible capacity	12 GiB

5. Workload

- AMF 논문에서 사용한 OLTP Workload를 선정

□ 논문에서 EXT4와 F2FS의 성능 차이가 나타났으며, 기본 Filebench Workload에도 포함되어 있어 비교 대상으로 OLTP를 선택

- 실험 환경 수정

□ I/O 단위: 2 KiB → 4 KiB (SSD Page Size에 맞게 변경)

- 작업 파일 : Data File : 9 GiB, Log File : 256 MiB

5. Workload

- SW (Sequential Write) : 랜덤 Delete □ 순차 쓰기를 수행
- RW (Random Write) : 랜덤 Delete □ 랜덤 쓰기를 수행
- SOW (Sequential Overwrite) : 기존 데이터를 순차적으로 Overwrite
- ROW (Random Overwrite) : 기존 데이터를 랜덤한 위치에 Overwrite
- ROW 1, 2, 3, 4를 수행하면서 File의 크기를 8MB에서 1MB로 줄이고, Overwrite 횟수를 달리하여 계속 실험을 수행
- OLTP : Database transaction Workload

5. Workload

1. 초기 데이터 10GB 순차 Write
2. 각 Workload 진행
3. User_write_pages / gc_write_pages 측정
4. FS WAF / FTL WAF / Total WAF 계산
5. EXT4, F2FS 비교

6. Expected Result

	EXT4+ PFTL	EXT4+ DFTL	F2FS+ PFTL		F2FS+ DFTL		AMF
	FTL	FTL	FS	FTL	FS	FTL	FS
FIO(SW)	1.00	1.00	1.00	1.00	1.00	1.00	1.00
FIO(RW)	1.41	1.45	1.35	1.82	1.34	2.18	1.38
Postmark(L)	1.00	1.00	1.00	1.00	1.00	1.00	1.00
Postmark(H)	1.12	1.35	1.17	2.23	1.18	2.89	1.16
Non-Trans	1.97	2.00	1.58	2.90	1.59	2.97	1.59
OLTP	1.45	1.46	1.23	1.78	1.23	1.79	1.24
TPC-C	2.33	2.21	1.81	2.80	1.82	5.45	1.87
DFSIO	1.0	1.0	1.0	1.0	1.0	1.0	1.0
TeraSort	1.0	1.0	1.0	1.0	1.0	1.0	1.0
WordCount	1.0	1.0	1.0	1.0	1.0	1.0	1.0

가설 1 : Write / Overwrite 방식에 따라 F2FS의 WAF가 달라질 것이다.

- SW의 경우에는 $WAF \approx 1$
- RW, ROW 같은 작업 수행 시 WAF가 증가할 것이다

가설 2 : F2FS의 이중 GC로 인하여 EXT4 대비 높은 WAF가 나타날 것이다

- FS, FTL에서의 이중 GC로 인하여 Write Amplification이 EXT4 대비 높게 나타날 것이다

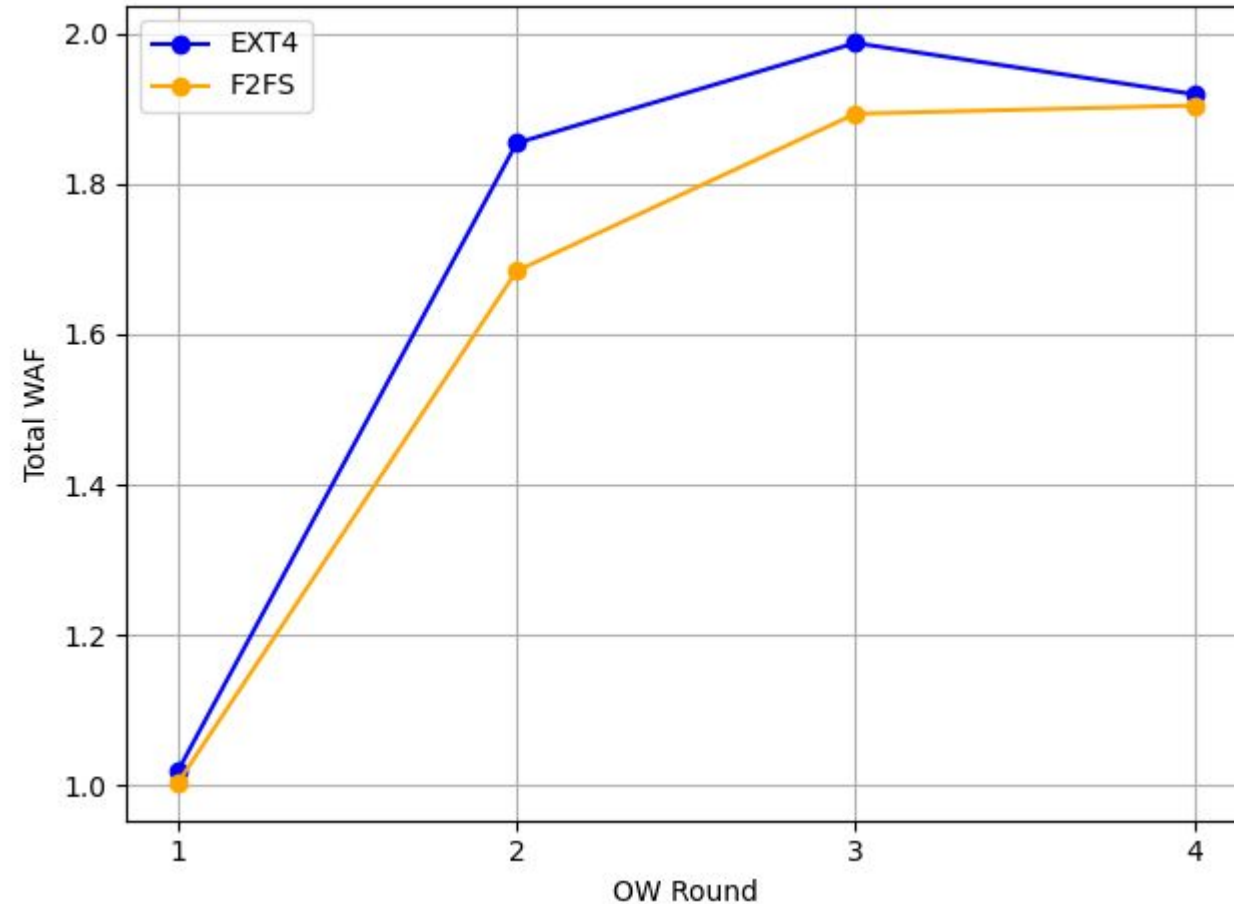
7. Experiment Result

	EXT4 SW	EXT4 RW	EXT4 SOW	EXT4 ROW	EXT4 OLTP	F2FS SW	F2FS RW	F2FS SOW	F2FS ROW	F2FS OLTP
FS WAF	1.013	1.015	1.013	1.0065	1.4052	1.004	1.004	1.002	1.0044	1.591
FTL WAF	1.164	1.164	1.000	1.9661	2.5416	1.151	1.155	1.000	1.8837	4.060
Total WAF	1.178	1.182	1.013	1.9789	3.5714	1.155	1.159	1.002	1.8920	6.459

7. Experiment Result - ROW2

10GiB 초기화 후 4GiB씩 쓰기 3번 반복 환경

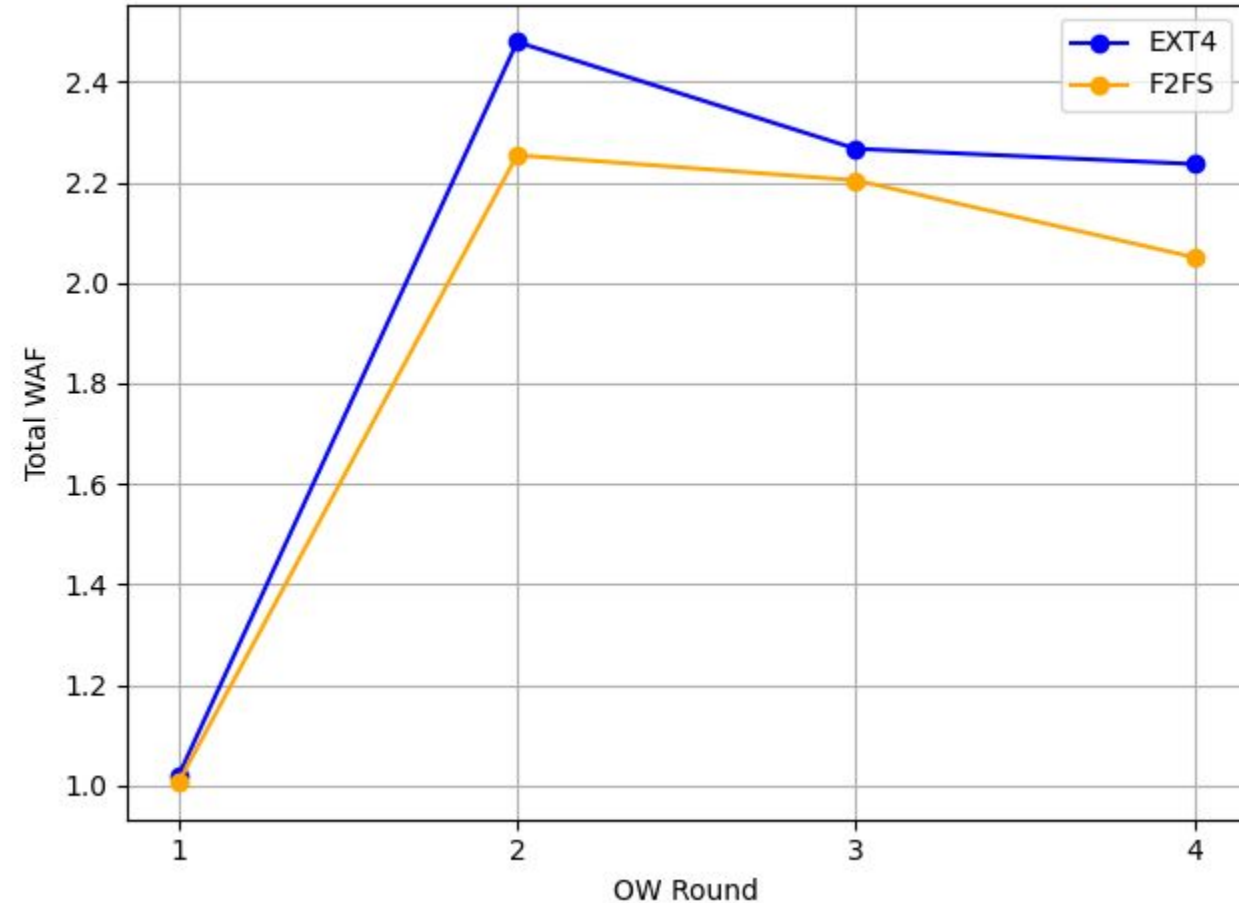
OverWrite시점에서 WAF가 크게 늘어나지만,
EXT4가 더 높은 WAF를 보인다



7. Experiment Result - ROW3

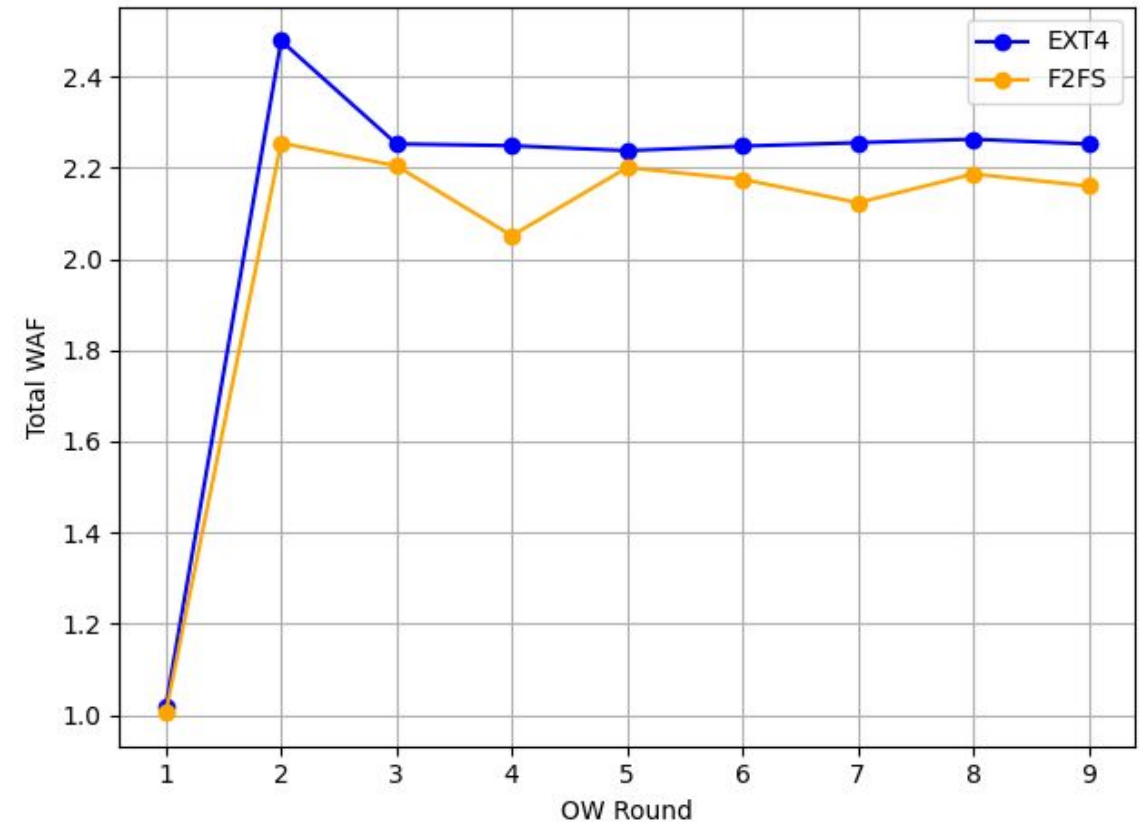
각 File의 크기를 작게 초기화한 후 동일 실험을 반복 환경

양쪽 File system 모두 기존보다 높은 WAF를 보였고, EXT4가 더 높은 WAF를 가졌다.



7. Experiment Result – ROW4

첫번째 GC 이후 두 FS의 WAF는 안정화 되지만,
F2FS 쪽이 더 변동폭이 크고, 기본적으로
EXT4쪽이 더 높은 곳에서 안정화 된다.

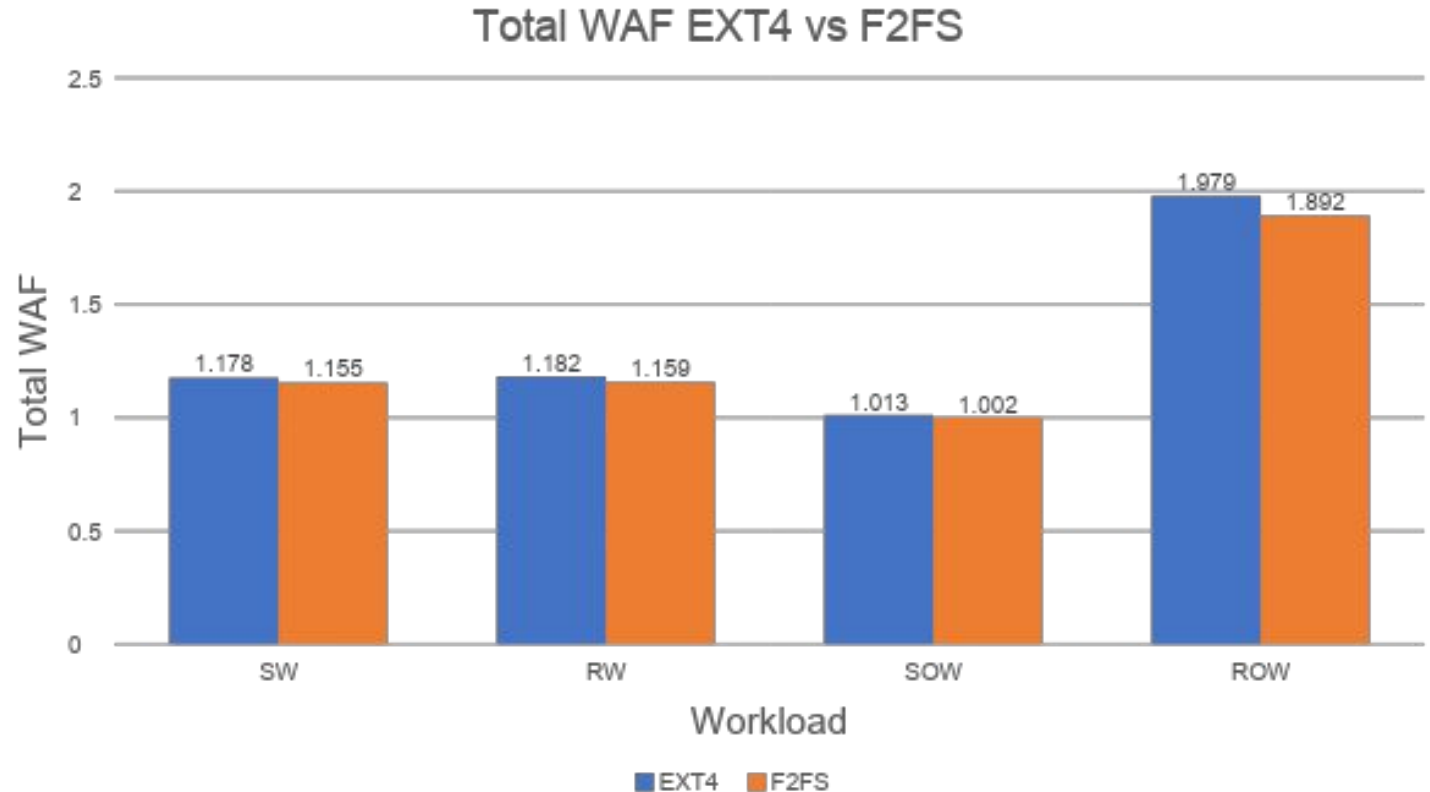


8. Analysis

가설 1 : Write / Overwrite 방식에 따라 F2FS의 WAF가 달라질 것이다.

- SW의 경우에는 WAF ≈ 1
- RW, Random Overwrite 작업 수행 시 WAF가 증가할 것이다

→ 가설 1은 실제 실험 결과와 일치

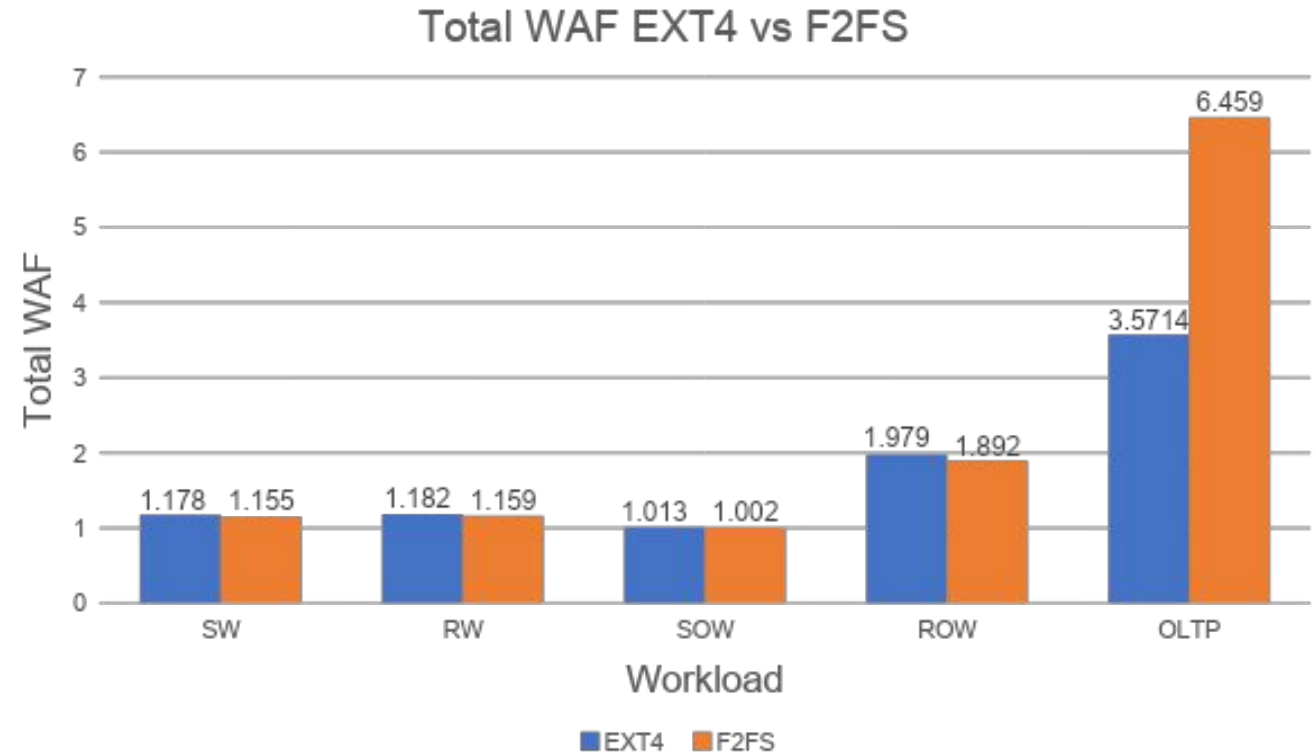


8. Analysis

가설 2 : F2FS의 이중 GC로 인하여 EXT4 대비 높은 WAF가 나타날 것이다

■ FS, FTL에서의 이중 GC로 인하여 Write Amplification이 EXT4 대비 높게 나타날 것이다

□ 가설 2는 실제 실험 결과와 반대로 나타났다



8. Analysis

- 이유 1 : F2FS의 Segment Cleaning에 의한 추가 Write가 제한적이었을 가능성
- 10GB를 미리 채운 상태에서 추가 10GB Write를 수행했으므로 F2FS의 Cleaning은 발생했을 것으로 예상
- 그러나 F2FS가 항상 모든 Segment를 Cleaning하는 것은 아니며, Cleaning 대상에서 Valid Page만 선택적 이동
- 따라서 Cleaning 과정에서 발생하는 Copy할 Valid Page가 많지 않았다면 추가 Write도 제한적

8. Analysis

- 이유 2 : EXT4의 FTL GC에 의한 추가 Write 발생 가능성

→ EXT4는 FS-level Cleaning을 수행하지 않으므로, Overwrite 수행 시 기존 Page가 Invalid Page가 됨

→ Invalid Page가 증가하면 FTL에서 GC 수행

→ GC 과정에서 Valid Page Copy가 발생하여 FTL WAF가 증가했을 가능성

8. Analysis

- 이유 3 : Workload에 따라 Cleaning 및 GC의 영향이 달라짐

F2FS는 여유 공간이 충분한경우, GC를 실행하지 않고, 기존의 빈 세그먼트를 찾아서 쓴다.

처음 쓰기는 순차 쓰기이기에, 초기화한 파일을 삭제 할때 Segment 단위로 삭제됐을 가능성 + 4GiB의 디스크 작업 공간이 존재

□ 따라서 GC가 트리거 되지 않았을 가능성이 존재

9. Conclusion & Future Work

- Workload에 따른 F2FS의 Write Amplification 특성을 확인하고, EXT4와 비교
- FEMU 환경에서 EXT4와 F2FS를 대상으로 SW, RW, SOW, ROW Workload를 수행하고, FS WAF, FTL WAF를 측정하여 Total WAF를 비교
- SW / RW / SOW / ROW Workload에서 F2FS의 Total WAF가 EXT4와 같거나 더 낮게 나타남
- 즉 F2FS가 이중으로 GC를 수행하더라도, 반드시 EXT4보다 높은 WAF를 보이는 것은 아니다.

9. Conclusion & Future Work

1. Disk 여유공간에 따른 WAF 변화 검증 및 재실험

□ 논문과 달랐던 실제 결과를 검증하기 위해 Disk의 여유공간을 줄이고 재실험 수행

2. ZNS 환경에서 Zone-based Mapping 효과 검증

□ F2FS의 Zone-based Mapping이 ZNS 환경에서도 WAF 감소 효과를 보이는지 확인

3. GC 발생량과 WAF의 관계 분석

□ GC 횟수 및 Valid Page Copy량을 측정하여 WAF 증가 원인을 검증

Thank you